# Facultad de Ciencias - UNAM
# Lógica Computacional 2026-2
# Práctica 4: Algoritmo de Unificación

Favio Ezequiel Miranda Perea  Patricio Ordoñez Blanco  Eduardo Vargas Pérez

7 de Abril de 2026
**Fecha de entrega:** 23 de Abril de 2026

## 1. Introducción

Computacionalmente hablando, un aspecto muy importante de la lógica de primer orden es la unificación porque proporciona una herramienta fundamental para saber cuándo dos expresiones pueden hacerse iguales mediante sustituciones, de modo que la idea intuitiva de hacer coincidir términos se convierte en un procedimiento formal que sirve como base para los motores inferencia (como se verá más adelante con `Prolog` o como el sistema de inferencia de tipos en `Haskell`).

## 2. Objetivo

Con esta práctica se busca que refuercen su conocimiento sobre sustituciones en términos al implementar el algoritmo Martelli-Montari para así tener una base sólida sobre el funcionamiento interno de cualquier sistema de inferencia.

## 3. Desarrollo de la práctica

### Términos

Primero, hay que recordar que los términos se definen recursivamente de la siguiente manera:

- Los símbolos de constante $c_1, ..., c_n, ...$ son términos.
- Las variables $x_1, ..., x_n, ...$ son términos.
- Si $f^{(m)}$ es un símbolo de función y $t_1, ..., t_m$ son términos entonces $f(t_1, ..., t_m)$ es un término.
- Son todos.

Siendo así que se necesita una forma de representar términos en `Haskell`. Para ello, se utilizará el siguiente sinónimo junto con el siguiente tipo de dato algebraico:

```haskell
type Nombre = String
data Term = Var Nombre | Fun Nombre [Term] deriving (Eq)
```

El sinónimo `Nombre` se usa para etiquetar algún término. Mientras que en el tipo de dato `Term` tenemos únicamente dos constructores; uno para las variables y otro para las funciones.
En un primer análisis parecería que falta una forma de representar las constantes. Sin embargo, se representarán precisamente como en `Haskell` para poder facilitar la implementación; es decir, como una función que no recibe nada. Por ejemplo:

- `Var "x"` representa la variable $x$.
- `Fun "a"[]` representa la constante $a$.
- `Fun "f"[Fun "g"[Var "x"], Var "y"]` representa la función $f(g(x), y)$

Adicionalmente, se usará el siguiente código para facilitar la lectura de los términos en la terminal:

```haskell
instance Show Term where
    show (Var p) = p
    show (Fun a []) = a
    show (Fun f args) = f ++ "(" ++ showArgs args ++ ")"

-- función auxiliar
showArgs :: [Term] -> String
showArgs [] = ""
showArgs [x] = show x
showArgs (x:xs) = show x ++ "," ++ showArgs xs
```

## Sustituciones

Para representar la sustitución $[\vec{x} := \vec{t}\,]$ se usarán listas de pares ordenados $[(x_1, t_1), ..., (x_n, t_n)]$. Para esto se define el sinónimo

```haskell
type Subst = [(Nombre,Term)]
```

### Ejercicio

- `apsubT :: Term -> Subst -> Term`

  Esta función debe implementar la aplicación de la sustitución $[\vec{x} := \vec{t}\,]$ sobre un término $t$; de modo que si $s$ implementa la sustitución $[\vec{x} := \vec{t}\,]$, entonces `apsubT r s` debe devolver el término $r[\vec{x} := \vec{t}\,]$ **[1 punto]**

  **Ejemplos**

  `ghci > apsubT (Var "x") [("x", Fun "a"[])]` ▷ `a`
  `ghci > apsubT (Fun "f"[Var "x", Var "y"]) [("x", a)]` ▷ `f(a,y)`

  ***Hint:*** Deberán implementar una función auxiliar que `[Term] -> Subst -> [Term]` que aplique una sustitución a una lista de términos.

## Algoritmo de unificación Martelli-Montanari

Para implementar el algoritmo de unificación hay que recordar las siguientes ideas para unificar primero dos términos $t_1$ y $t_2$:

- Si los términos $t_1$ y $t_2$ son constantes, entonces serán unificables si y sólo si $t_1 = t_2$.
- Si alguno de los términos es una variable, digamos $t_1 = x$, entonces si $x$ figura en $t_2$ entonces los términos no son unificables, en caso contrario $[x := t_2]$ es un unificador.
- Si $t_1$ y $t_2$ son términos funcionales, entonces son unificables si y sólo si se cumplen las siguientes:
  - Los símbolos de función de $t_1$ y $t_2$ son el mismo.
  - Cada par de términos correspondiente en los argumentos son unificables.

Una vez teniendo una forma de unificar dos términos, se puede generalizar la idea para verificar recursivamente si un conjunto de términos $W = \{t_1, t_2, ..., t_n\}$ es unificable:

- Hallar $\mu_1$ un unificador para $t_1$ y $t_2$
- Hallar $\mu_2$ un unificador para $[t_2\mu_1, t_3\mu_1, ..., t_n\mu_1]$
- Un unificador para $W$ es $\mu_1\mu_2$

Nótese que la unificación se hace unificando 2 términos para obtener un unificador y usarlo para tratar de unificar un conjunto con un término menos. De este modo, si alguna unificación falla, el conjunto $W$ no es unificable.

## Implementación del algoritmo de unificación

- `simpSus :: Subst -> Subst`
  Esta función debe simplificar una sustitución eliminando toda sustitución trivial de la forma $x := x$ **[1 punto]**

  **Ejemplos**

  ```
  ghci > simpSus [("x", Fun "f" [Var "x"])] ▷ [("x", f(x))]
  ghci > simpSus [("x", Var "x"), ("y", a)] ▷ [("y", a)]
  ```

- `compSus :: Subst -> Subst -> Subst`
  Dadas dos sustituciones $\sigma_1$ y $\sigma_2$, esta función debe devolver la composición $\sigma_1\sigma_2$ **[2 puntos]**

  **Ejemplos**

  ```
  ghci > compSus [("x", Fun "f"[y])] [("y", a)] ▷ [("x",f(a)),("y",a)]
  ghci > compSus [("x", a)] [("x", b)] ▷ [("x", a)]
  ```

- `unifica :: Term -> Term -> [Subst]`
  Dados dos términos $t_1$ y $t_2$, la función debe devolver una lista que tiene como único elemento un unificador de máxima generalidad para $t_1$ y $t_2$ si es que son unificables. Si $t_1$ y $t_2$ no son unificables, la función debe devolver una lista vacía. **[3 puntos]**

  **Ejemplos**

  ```
  ghci > unifica (Fun "f"[x]) (Fun "f"[x, y]) ▷ []
  ghci > unifica (Fun "f"[x, y]) (Fun "f"[a, b]) ▷ [[("x", a), (”y", b)]]
  ```

  ***Hint:*** Deberán implementar una función auxiliar `unificaListas :: [Term] -> [Term] -> [Subst]` que unifique dos listas de términos de la misma longitud componente a componente. Es decir, `unificaListas [s1,..,sk] [r1,..,rk]` devuelve la lista $[\mu]$ con un unificador tal que $s_j\mu = r_j\mu$ para toda $1 \leq j \leq k$. Si algún par de términos no son unificables, se debe devolver la lista vacía.

- `unificaConj :: [Term] -> [Subst]`
  Dado un conjunto de términos $W$ (representado como una lista de términos), esta función debe devolver un unificador de máxima generalidad para $W$. Si $W$ no es unificable, se debe devolver la lista vacía. **[3 puntos]**

  **Ejemplos**

  ```
  ghci > unificaConj [Fun "Q"[y,z], Fun "Q"[x, Fun "f"[a]], Fun "Q"[Fun "f"[z], z]]
  ▷ [[(”y",f(f(a))),("z",f(a)),("x",f(f(a)))]]
  ghci > unificaConj [Fun "f"[x,y,x], Fun "f"[y,Fun "g"[x],x],Fun "f"[a,z,a]]
  ▷ []
  ```

¡Buena suerte a todos! ☺☺☺

**Nota:** Para mayor referencia teórica sobre los conceptos de esta práctica, pueden revisar las notas 5,7 y 11 del curso.